



LAYOUT Y DISEÑO CON TAILWIND CSS

Objetivos

Al finalizar este tema, los alumnos podrán:

- Construir layouts usando Flexbox y Grid
- Controlar ancho, alto y contenedores
- Usar posicionamiento (`relative`, `absolute`, `fixed`)
- Manejar overflow y z-index
- Aplicar estos conceptos con Tailwind CSS 4

Sistema Flexbox

- Flexbox = modelo para **alinear y distribuir elementos en una fila o columna**
- Clases principales:

Clase	Función
<code>flex</code>	Activa flexbox
<code>flex-row</code>	Dirección horizontal (default)
<code>flex-col</code>	Dirección vertical
<code>justify-start/center/end/between</code>	Alineación horizontal
<code>items-start/center/end/stretch</code>	Alineación vertical

Ejercicio 1

Crea un contenedor con tres cajas en fila:

- Clase `flex`
- Dirección horizontal (`flex-row`)
- Alinea elementos al centro vertical y horizontal (`justify-center items-center`)
- Cada caja con fondo distinto y padding `p-4`

```
<div class="flex justify-center items-center gap-4 h-32 bg-gray-100">  
  <div class="bg-red-200 p-4">Caja 1</div>  
  <div class="bg-green-200 p-4">Caja 2</div>  
  <div class="bg-blue-200 p-4">Caja 3</div>  
</div>
```

- ♦ Modifica `flex-col` y observa el cambio

Grid layout

- Grid = modelo de **rejilla para filas y columnas**
- Clases principales:

Clase	Función
<code>grid</code>	Activa grid
<code>grid-cols-2/3/4</code>	Número de columnas
<code>grid-rows-2/3</code>	Número de filas
<code>gap-4</code>	Espacio entre celdas

Ejercicio 2

Crea un contenedor grid de 2 columnas y 2 filas:

```
<div class="grid grid-cols-2 grid-rows-2 gap-4 bg-gray-100 p-4">  
  <div class="bg-red-200 p-4">1</div>  
  <div class="bg-green-200 p-4">2</div>  
  <div class="bg-blue-200 p-4">3</div>  
  <div class="bg-yellow-200 p-4">4</div>  
</div>
```

- ♦ Cambia el número de columnas y filas para experimentar

Ancho, alto y contenedores

- Clases de ancho y alto:
- Contenedor centrado: `container mx-auto`

Clase	Función
<code>w-32</code>	ancho 8rem
<code>h-16</code>	alto 4rem
<code>w-full</code>	ancho completo
<code>h-screen</code>	alto pantalla

Ejercicio 3

Crea un `div` con:

- `w-64 h-32`
 - Fondo azul `bg-blue-200`
 - Centrado en pantalla `mx-auto my-8`
- ♦ Cambia `w-full` y `h-screen` y observa la diferencia

Posicionamiento: relative, absolute, fixed

`relative` → posición relativa al flujo normal

`absolute` → posición absoluta dentro del padre relativo

`fixed` → posición fija respecto a la ventana

Ejemplo:

```
<div class="relative h-32 bg-gray-100">  
  <div class="absolute top-0 left-0 bg-red-200 p-2">Absolute</div>  
</div>
```

Ejercicio 4

Crea un contenedor `relative h-32 bg-gray-100`

Añade un `div absolute top-0 right-0 bg-red-200 p-2`

Añade un `div absolute bottom-0 left-0 bg-blue-200 p-2`

Observa cómo se posicionan los elementos dentro del contenedor

Overflow y z-index

- **Overflow** controla lo que pasa si el contenido se sale del contenedor:
- **z-index** controla la superposición:

Clase	Función
<code>overflow-auto</code>	scroll si hay exceso
<code>overflow-hidden</code>	oculta contenido extra
<code>overflow-scroll</code>	siempre scroll

Clase	Función
<code>z-10</code>	más arriba
<code>z-0</code>	base
<code>z-50</code>	muy arriba

Ejercicio 5

Crea un contenedor `relative w-64 h-32 overflow-auto bg-gray-100 p-2`

Añade varios `div` grandes dentro para que haya scroll

Prueba `overflow-hidden` y `overflow-scroll`

Añade dos elementos `absolute` con `z-10` y `z-20` para ver superposición

Resumen

Flexbox: `flex`, `flex-row/col`, `justify-`, `items-`

Grid: `grid`, `grid-cols-`, `grid-rows-`, `gap-`

Dimensiones: `w-`, `h-`, `container` `mx-auto`

Posicionamiento: `relative`, `absolute`, `fixed`

Overflow: `overflow-auto/hidden/scroll`

Z-index: `z-0/10/50`